

확률과 확률분포

확률

- 확률(probability)
 - 어떤 사건(event)이 일어날 가능성
 - 예.
 - 동전 던지기에서 앞면이 나올 가능성
 - 주사위 던지기에서 특정 눈금이 나올 확률
 - 주식투자에서 이득을 볼 가능성
- 의사결정
 - 확실성(certainty) 보다는 불확실한 상황에서 이루어지는 경우가 많음
 - 미래에 대한 불확실성의 정도가 더욱 심하다고 할 수 있음
 - 불확실성 하에서 의사결정의 오류를 줄이기 위해 확률에 대한 연구 필요

확률의 기본

- 표본공간과 표본점
- 사상(사건)
- 확률의 정의
 - 객관적 확률(고전적/경험적확률)
 - 주관적 확률
 - 공리적 확률
- 합사건 / 곱사건
- 상호배반
- 종속사건과 독립사건
- 결합확률 / 주변확률
- 조건부 확률
- 분할
- 전확률의 정리
- 베이즈 정리

확률변수와 확률분포

확률변수(random variable)

- 실험이나 관찰의 결과 값을 1:1 실수로 대응시키는 함수
- 일정한 확률로 나타나는(발생하는) 사건에 대해 숫자를 부여한 변수

확률변수 구분

- 이산 확률변수(discrete random variable) : 변수가 취할 수 있는 값의 개수가 유한
- 연속 확률변수(continuous random variable) : 변수가 취할 수 있는 값의 개수가 무한

확률분포(probability distribution)

- 확률변수가 취할 수 있는 모든 값에 대해 각각의 확률을 대응시킨 것

확률분포 구분

- 확률질량함수(probability mass function: pmf) : 확률변수가 이산형인 경우
- 확률밀도함수(probability density function: pdf) : 확률변수가 연속형인 경우

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
print(f'numpy: {np.__version__}, pandas: {pd.__version__}')
```

numpy: 2.5.3, pandas: 3.0.5

```
In [2]: %precision 3
```

Out[2]: '%.3f'

1) 공정한 주사위의 확률분포를 구하는 실험

```
In [3]: dice = [1,2,3,4,5,6]
prob = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6]
prob
```

Out[3]: [0.167, 0.167, 0.167, 0.167, 0.167, 0.167]

```
In [4]: n_trial = 100
sample = np.random.choice(dice, size=n_trial, p=prob)
print(sample)

freq, bins = np.histogram(sample, bins=6, range=(1,7))
print(freq, bins)

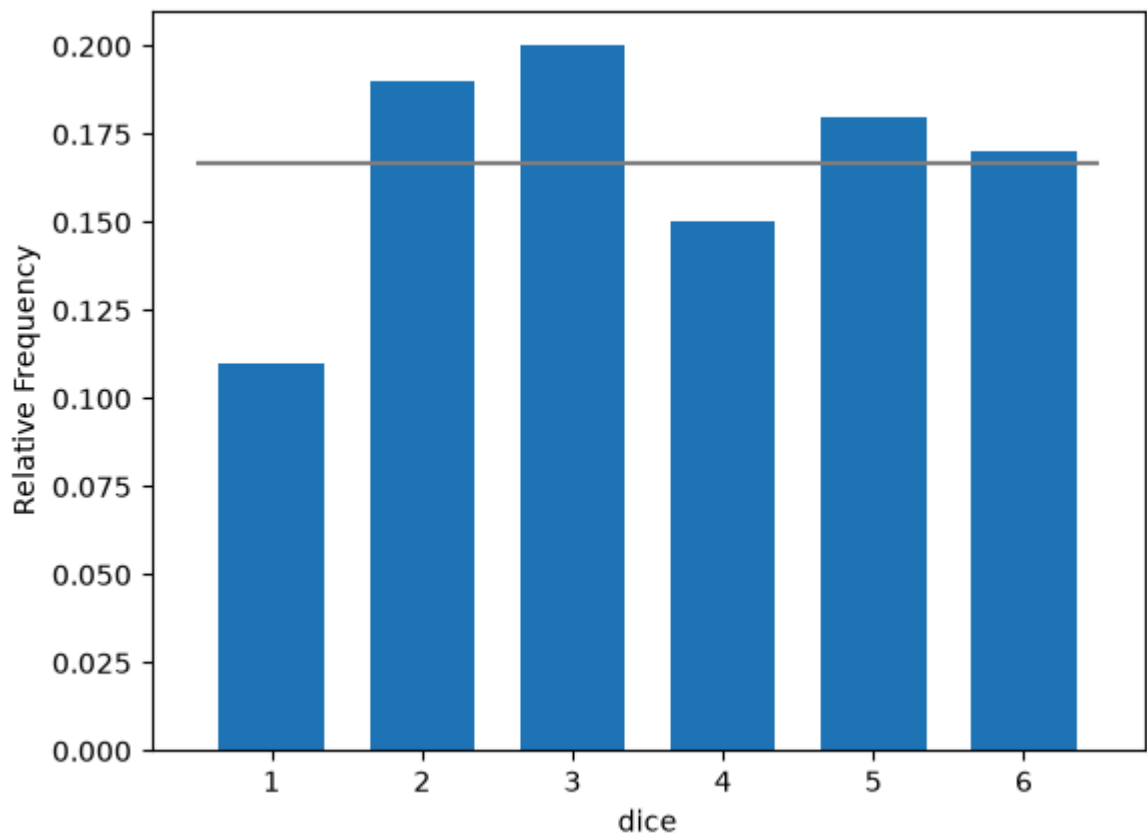
dice_df = pd.DataFrame({'Freq':freq, 'Rel_Freq':freq/n_trial},
                        index=pd.Index(np.arange(1,7), name='dice'))
dice_df
```

```
[5 4 5 6 2 3 3 4 4 5 4 3 2 6 4 3 3 2 3 3 5 5 2 2 2 5 3 2 2 3 5 5 4 5 4 6 4
 6 4 1 2 6 6 4 2 3 3 1 2 5 5 3 6 2 1 2 3 4 5 6 6 6 3 2 1 6 2 3 1 5 1 5 2 6
 3 4 3 3 2 1 5 3 5 2 4 2 1 1 1 5 4 1 6 6 6 6 3 5 6 4]
[11 19 20 15 18 17] [1. 2. 3. 4. 5. 6. 7.]
```

Out[4]: Freq Rel_Freq

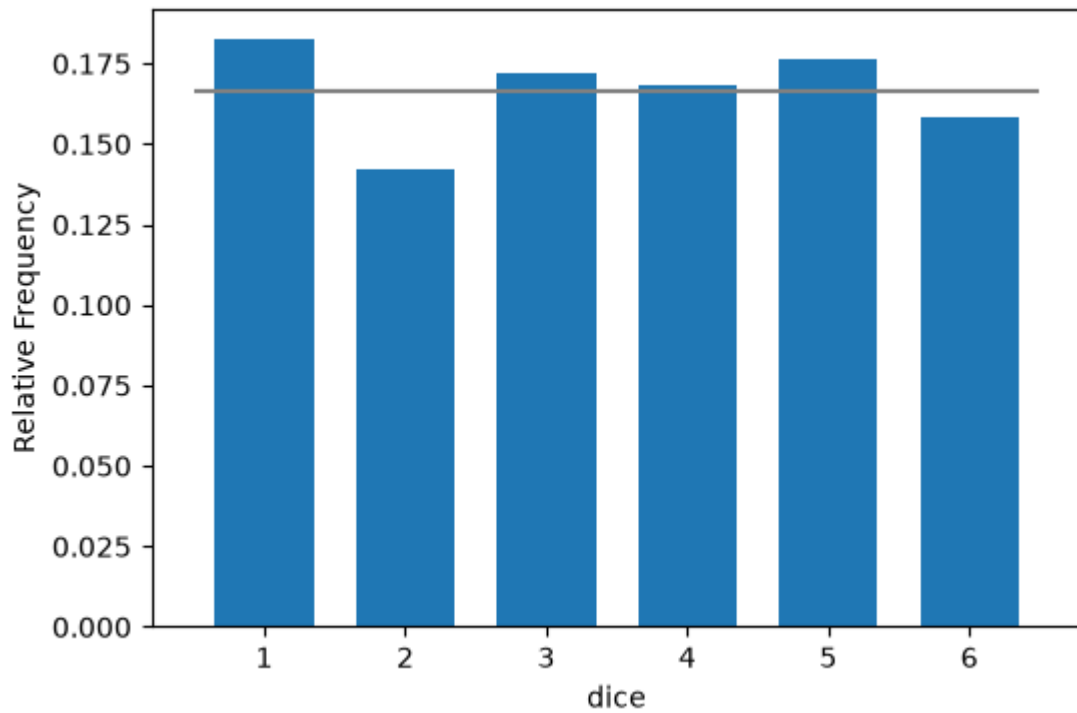
dice		
1	11	0.11
2	19	0.19
3	20	0.20
4	15	0.15
5	18	0.18
6	17	0.17

```
In [5]: plt.hist(sample, bins=6, range=(1,7), density=True, rwidth=0.7)
plt.xticks(np.linspace(1.5, 6.5, 6), [f'{x}' for x in range(1,7)])
plt.hlines(prob, np.arange(1,7), np.arange(2,8), color='gray')
plt.xlabel('dice')
plt.ylabel('Relative Frequency')
plt.show()
```



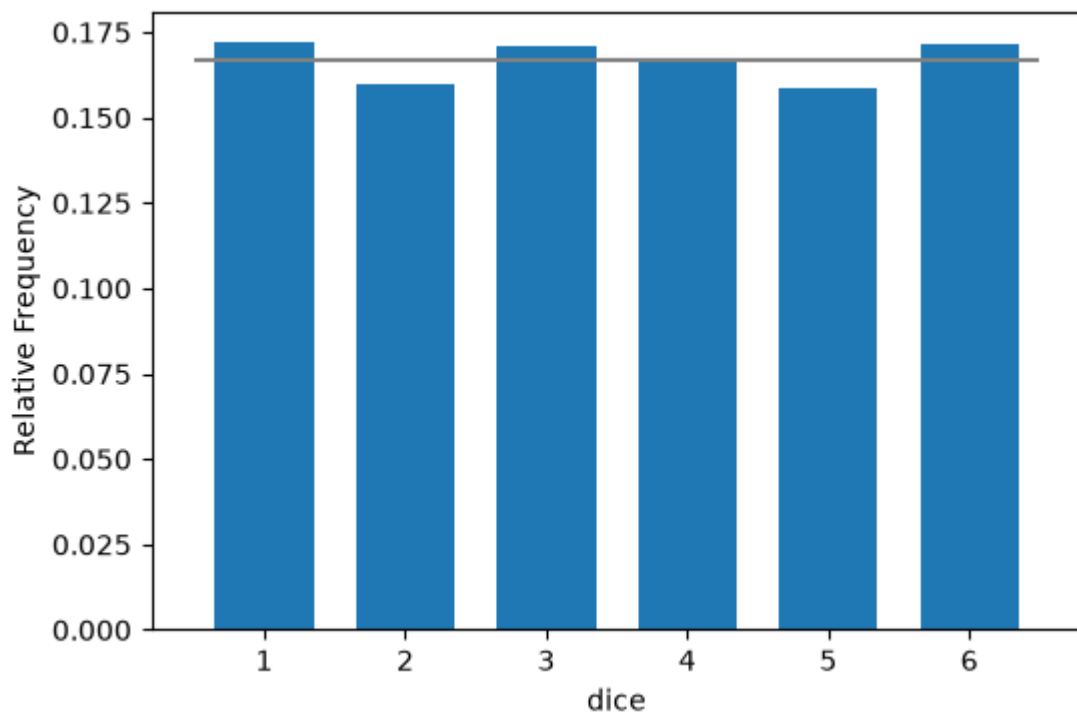
```
In [6]: def draw_dice_hist(num_trial, prob):
dice = [1,2,3,4,5,6]
sample = np.random.choice(dice, size=num_trial, p=prob)
plt.figure(figsize=(6,4))
plt.hist(sample, bins=6, range=(1,7), density=True, rwidth=0.7)
plt.xlabel('dice')
plt.ylabel('Relative Frequency')
plt.xticks(np.linspace(1.5, 6.5, 6), [f'{x}' for x in range(1,7)])
plt.hlines(prob, np.arange(1,7), np.arange(2,8), color='gray')
plt.show()
```

```
In [7]: draw_dice_hist(num_trial=2000, prob=[1/6]*6)
```

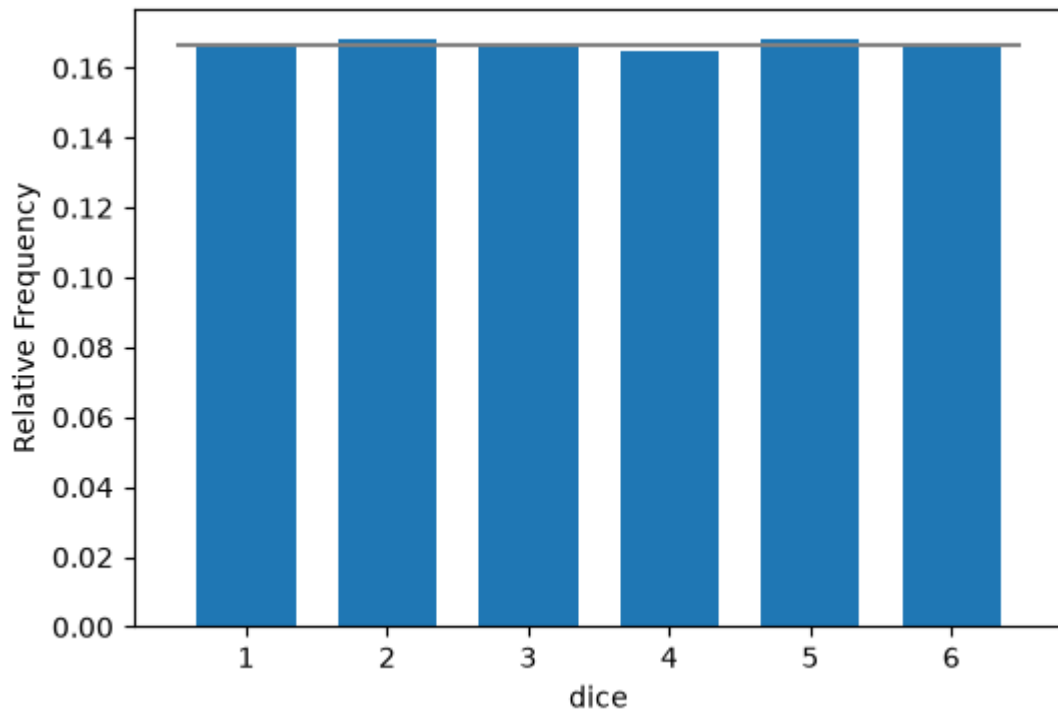


10000번 시도 : 실제 확률분포에 가까워짐

```
In [8]: draw_dice_hist(num_trial=10000, prob=[1/6]*6)
```



```
In [9]: draw_dice_hist(num_trial=100000, prob=[1/6]*6)
```



- 모평균(기대값)

```
In [10]: mu = 0
for x, p in zip(dice, prob):
    mu += x*p
mu
```

Out[10]: 3.500

```
In [11]: np.sum([x*p for x, p in zip(dice, prob)])
```

Out[11]: 3.500

```
In [12]: np.dot(dice, prob)
```

Out[12]: 3.500

- 표본평균

```
In [13]: dice_df
```

Out[13]: Freq Rel_Freq

dice		
1	11	0.11
2	19	0.19
3	20	0.20
4	15	0.15
5	18	0.18
6	17	0.17

```
In [14]: np.sum(dice_df.index * dice_df.Rel_Freq)
```

Out[14]: 3.610

2) 불공정한 주사위의 확률분포를 구하는 실험

```
In [15]: prob = [x/21 for x in range(1,7)]  
prob
```

Out[15]: [0.048, 0.095, 0.143, 0.190, 0.238, 0.286]

```
In [16]: np.sum(prob)
```

Out[16]: 1.000

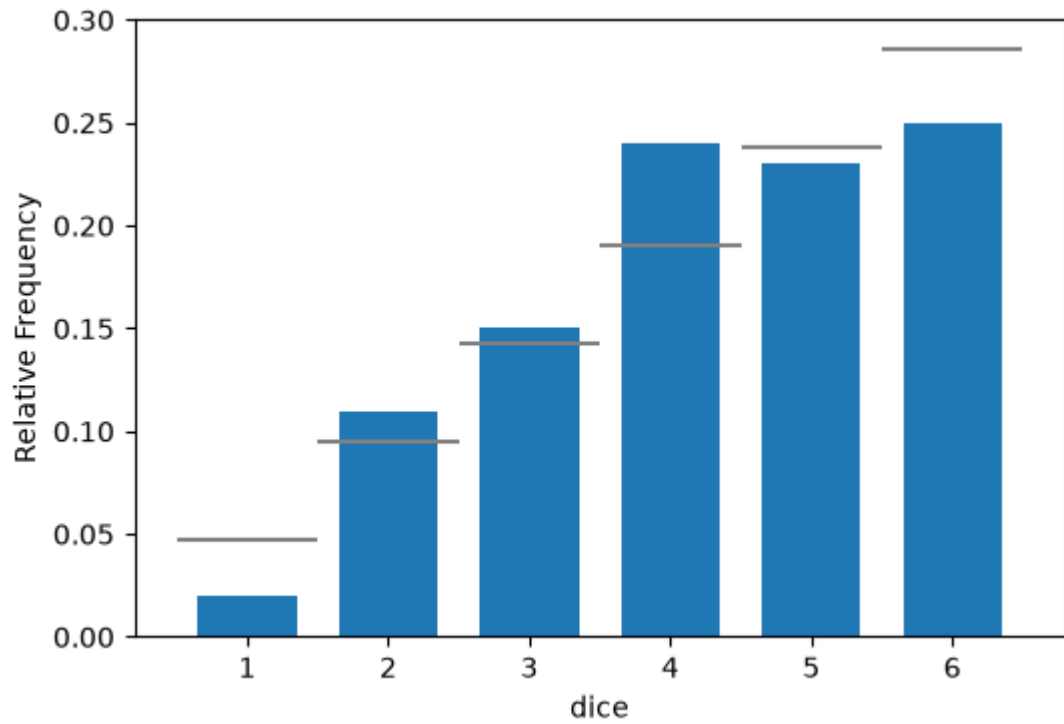
```
In [17]: def make_dice_df(n_trial, prob):  
    dice = [1,2,3,4,5,6]  
    sample = np.random.choice(dice, size=n_trial, p=prob)  
    freq, bins = np.histogram(sample, bins=6, range=(1,7))  
    dice_df = pd.DataFrame({'Freq':freq, 'Rel_Freq':freq/n_trial},  
                           index=pd.Index(np.arange(1,7), name='dice'))  
    return dice_df
```

```
In [18]: make_dice_df(n_trial=5000, prob=prob)
```

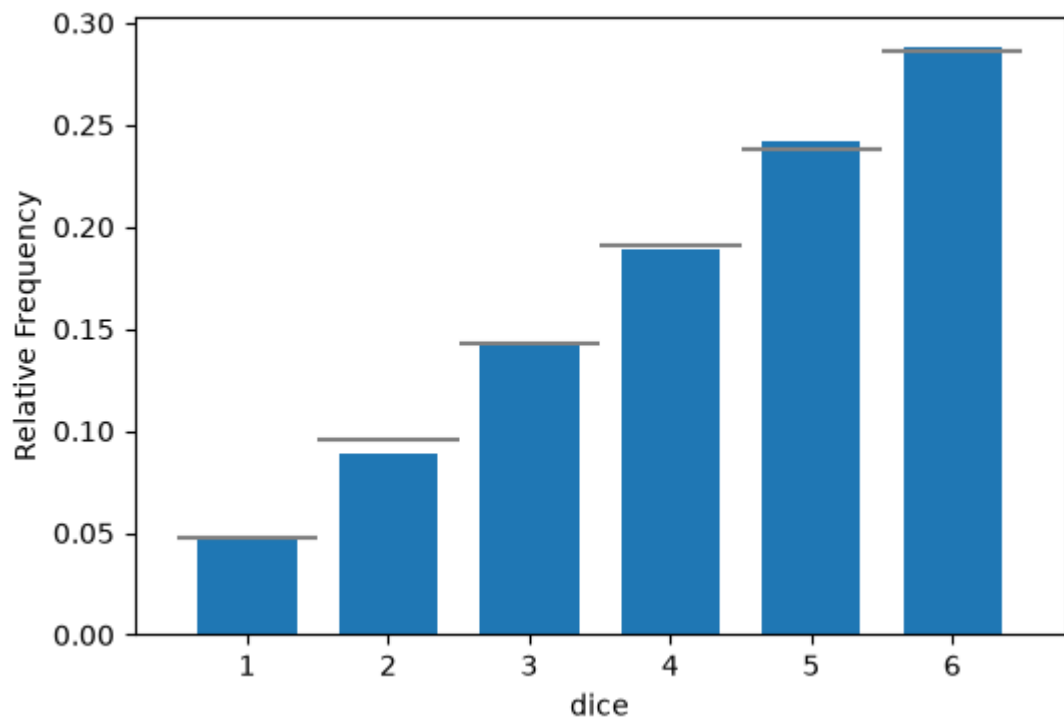
Out[18]: Freq Rel_Freq

dice		
1	243	0.0486
2	529	0.1058
3	708	0.1416
4	919	0.1838
5	1158	0.2316
6	1443	0.2886

```
In [20]: draw_dice_hist(num_trial=100, prob=prob)
```

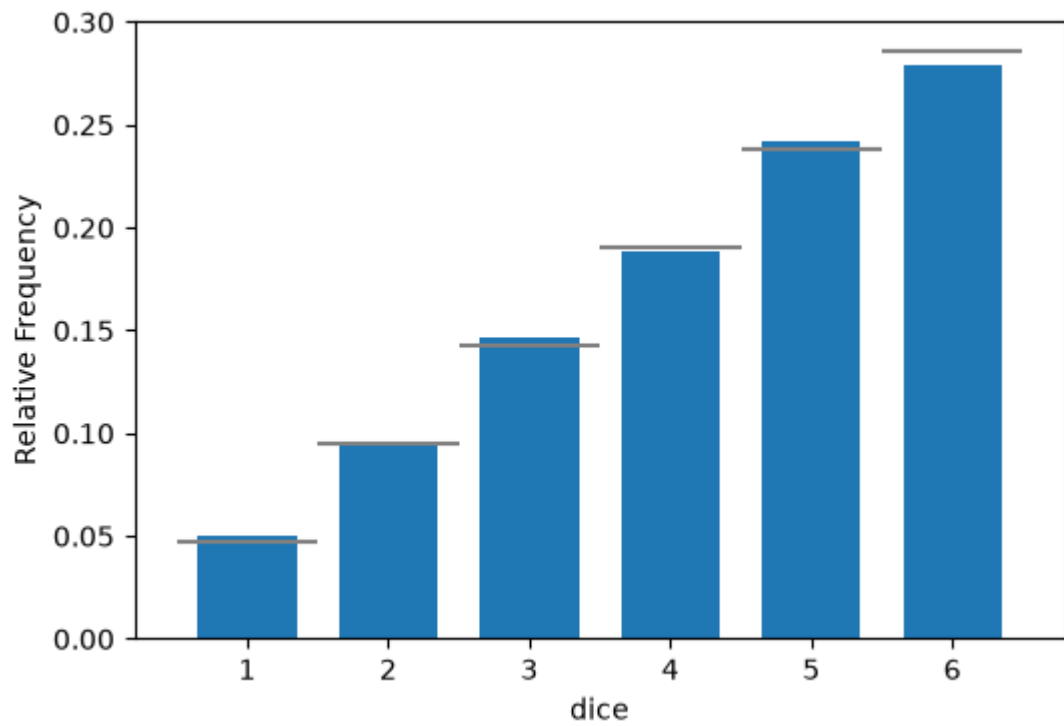


```
In [19]: draw_dice_hist(num_trial=5000, prob=prob)
```

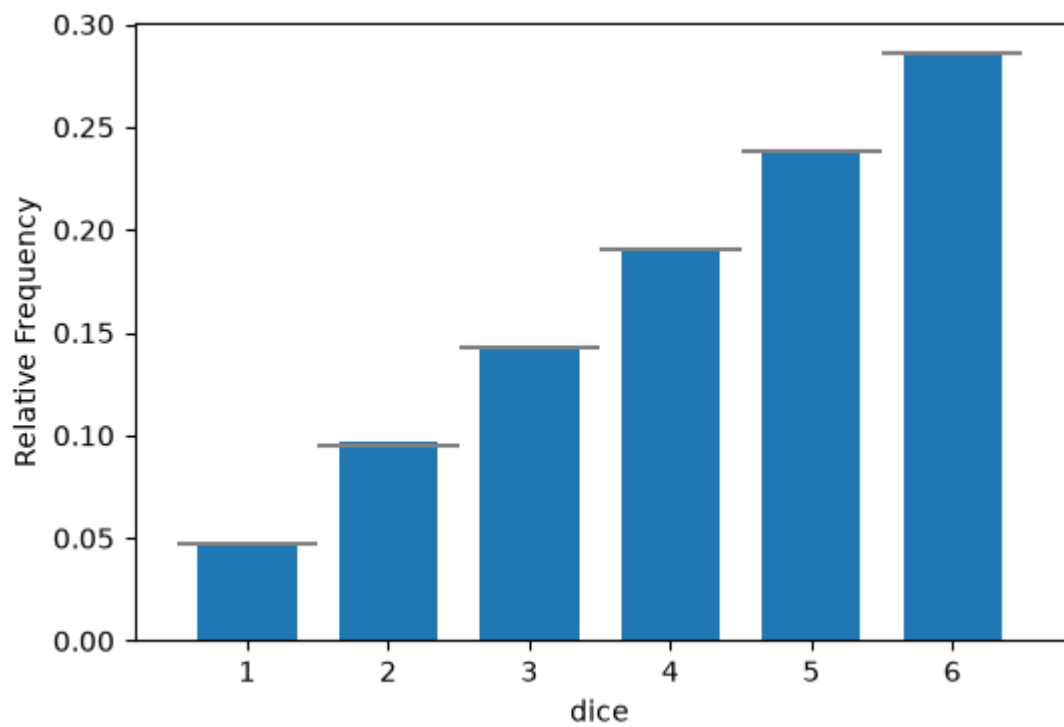


10000번 시도 : 실제 확률분포에 가까워짐

```
In [21]: draw_dice_hist(num_trial=10000, prob=prob)
```



```
In [22]: draw_dice_hist(num_trial=100000, prob=prob)
```



- 모평균(기대값)

```
In [23]: np.dot(dice, prob)
```

```
Out[23]: 4.333
```

- 표본평균


```
In [24]: dice_df = make_dice_df(n_trial=100, prob=prob)
         dice_df
```

```
Out[24]:
```

	Freq	Rel_Freq
--	------	----------

dice		
1	3	0.03
2	9	0.09
3	19	0.19
4	12	0.12
5	24	0.24
6	33	0.33

```
In [25]: np.dot(dice_df.index, dice_df.Rel_Freq)
```

```
Out[25]: 4.440
```

확률분포의 평균(mean)과 분산(variance)

확률분포의 평균(mean)

- 기대값(expected value)
 - 확률 변수의 기대값(E)은 각 사건이 벌어졌을 때의 이득과 그 사건이 벌어질 확률을 곱한 것을 전체 사건에 대해 합한 값
- 어떤 확률적 사건에 대한 평균의 의미
- $E(X)$ 또는 μ_X 로 표시
- 이산확률분포의 기대값 : 확률을 가중값으로 사용한 가중평균
- 연속확률분포의 기대값 : 적분개념의 면적

구분	이산확률분포	연속확률분포
기댓값	$E(X) = \mu_X = \sum_{\text{모든 } X}^n X_i \cdot P(X_i)$	$E(X) = \mu_X = \int_{-\infty}^{+\infty} x_i \cdot f(x_i) dx$

- 모평균(population mean) : μ
 - 모집단의 평균

```
In [26]: x_set = np.array([1,2,3,4,5,6])
```

```
# 확률질량함수(pmf)
def f(x):
    if x in x_set:
        return 1/6
    else:
        return 0
```

```
In [27]: f(1), f(10)
```

```
Out[27]: (0.167, 0)
```

```
In [28]: list(map(f, x_set))
```

```
Out[28]: [0.167, 0.167, 0.167, 0.167, 0.167, 0.167]
```

기대값 함수

```
In [29]: def E(X):
    x_set, f = X
    return np.sum([xi*f(xi) for xi in x_set])
```

```
In [30]: X=[x_set, f]
E(X)
```

```
Out[30]: 3.500
```

```
In [31]: def E2(X, g=lambda x:2*x +3):
    x_set, f = X
    return np.sum([g(xi)*f(xi) for xi in x_set])
```

```
In [32]: E2(X)
```

```
Out[32]: 10.000
```

확률분포의 분산(variance)

- $\text{Var}(X)$ 표시
- 확률변수의 평균(기대값)으로부터의 편차의 제곱에 대한 기대값으로 계산

구분	이산확률분포	연속확률분포
분산	$\begin{aligned} \text{Var}(X) &= \sum_{\text{모든 } X}^n [X_i - \mu_x]^2 P(X_i) \\ &= \sum_{\text{모든 } X}^n X_i^2 \cdot P(X_i) - \mu_x^2 \end{aligned}$	$\begin{aligned} \text{Var}(X) &= \int_{-\infty}^{+\infty} [x - \mu_x]^2 f(x) dx \\ &= \int_{-\infty}^{+\infty} x^2 f(x) dx - \mu_x^2 \end{aligned}$

```
In [33]: def V(X):
    x_set, f = X
    mu = E(X)
    return np.sum([(xi-mu)**2 * f(xi) for xi in x_set])
```

```
In [34]: X
```

Out[34]: [array([1, 2, 3, 4, 5, 6]), <function __main__.f(x)>]

In [35]: `V(X)`

Out[35]: 2.917

In [38]: `# V(X) = E(X^2) - (E(X))^2
np.dot(dice, dice)/6 - E(X)**2`

Out[38]: 2.917

In [39]: `# 불공정한 주사위에 대한 확률분포
def f2(x):
 if x in x_set:
 return x/21
 else:
 return 0`

In [40]: `x_set`

Out[40]: array([1, 2, 3, 4, 5, 6])

In [41]: `f2(6)`

Out[41]: 0.286

In [42]: `# 불공정한 주사위에 대한 확률분포에서 확률변수의 기대값
X2 = [x_set, f2]
E(X2)`

Out[42]: 4.333

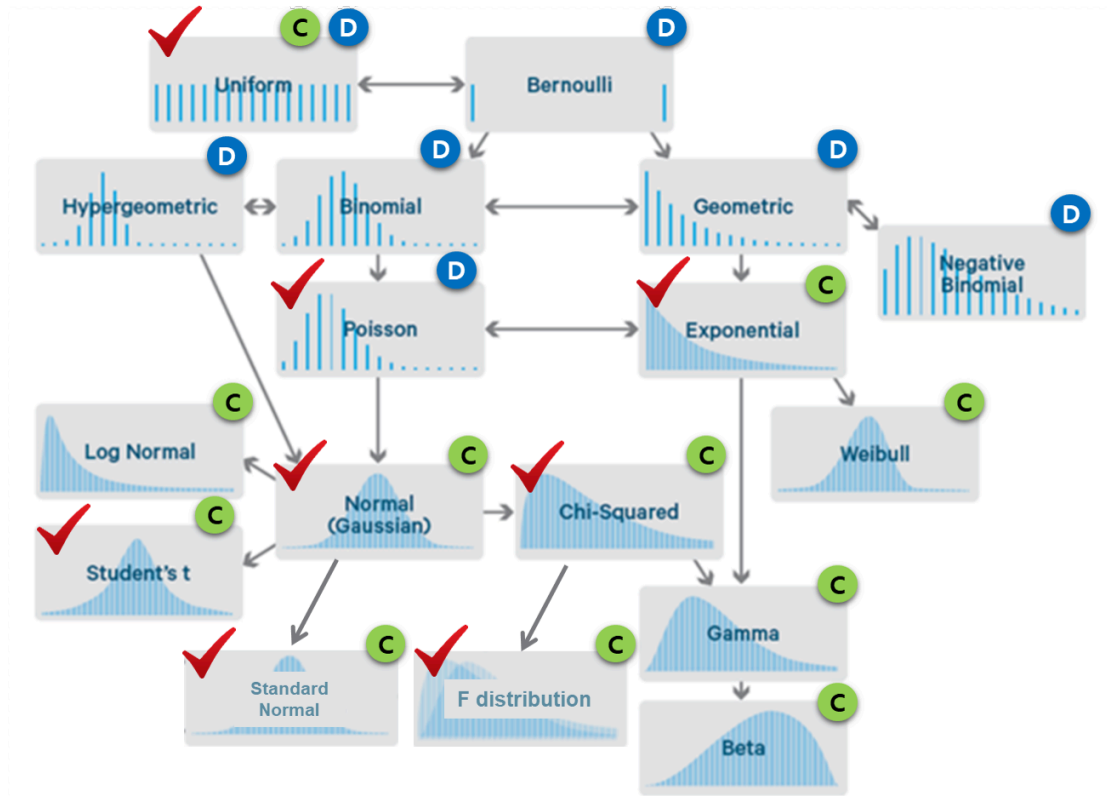
In [43]: `# 불공정한 주사위에 대한 확률분포에서 확률변수의 분산
V(X2)`

Out[43]: 2.222

In [44]: `import scipy
scipy.__version__`

Out[44]: '1.18.0'

- <https://scipy.org/>
- <https://docs.scipy.org/doc/scipy/reference/index.html>
- <https://docs.scipy.org/doc/scipy/reference/stats.html>



출처: <https://www.datasciencecentral.com/profiles/blogs/common-probability-distributions-the-data-scientist-s-crib-sheet>

```
In [45]: from scipy import stats
```

이산형 확률분포

- 베르누이 분포
- 이항분포
- 포아송분포
- 기하분포

이항분포(binomial distribution)

- $X \sim B(n, p)$

```
In [46]: import math
math.comb(10, 2)
```

Out[46]: 45

```
In [47]: # X ~ B(10, 0.5)
n, p = 10, 0.5
[math.comb(n, x) * p**x * (1-p)**(n-x) for x in range(n+1)]
```

Out[47]: [0.001, 0.010, 0.044, 0.117, 0.205, 0.246, 0.205, 0.117, 0.044, 0.010, 0.001]

```
In [48]: def binom_pmf(n,p):  
         return {x:math.comb(n, x) * p**x * (1-p)**(n-x) for x in range(n+1)}
```

```
In [49]: binom_pmf(n,p)
```

```
Out[49]: {0: 0.001,  
          1: 0.010,  
          2: 0.044,  
          3: 0.117,  
          4: 0.205,  
          5: 0.246,  
          6: 0.205,  
          7: 0.117,  
          8: 0.044,  
          9: 0.010,  
          10: 0.001}
```

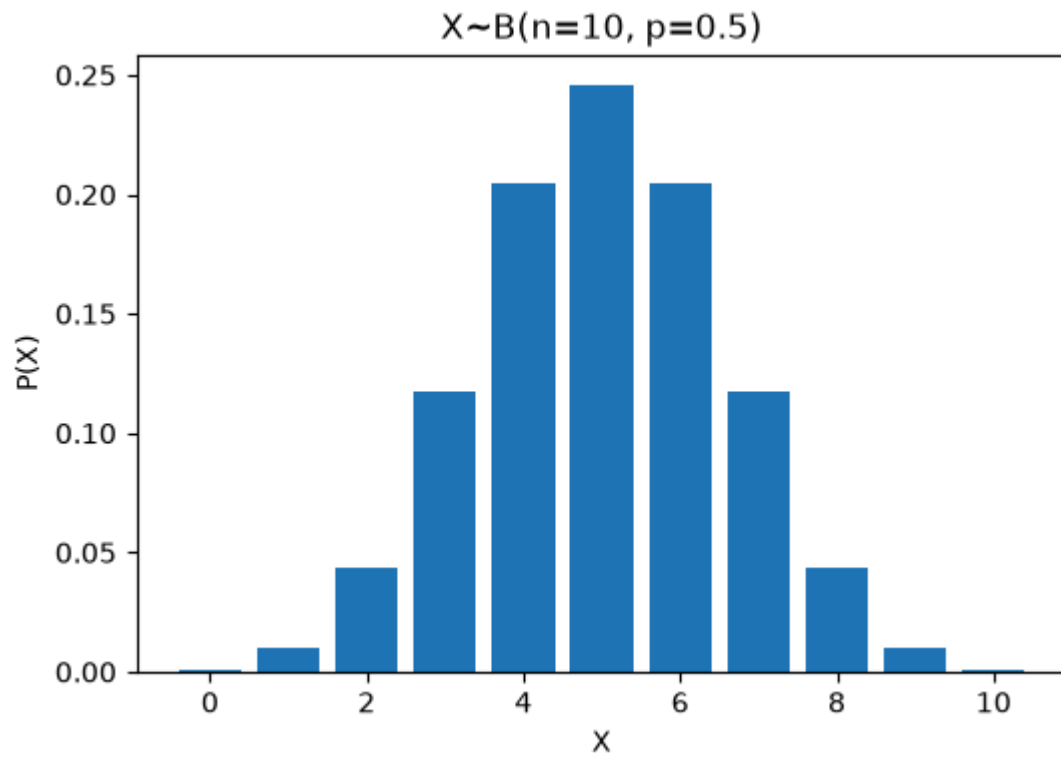
```
In [52]: df = pd.DataFrame(pd.Series(binom_pmf(n,p)), columns=['P(X)'])  
         df.index.name = 'X'  
         df
```

```
Out[52]:
```

	P(X)
X	
0	0.000977
1	0.009766
2	0.043945
3	0.117188
4	0.205078
5	0.246094
6	0.205078
7	0.117188
8	0.043945
9	0.009766
10	0.000977

```
In [54]: def draw_binom_pmf(n, p):  
         px = binom_pmf(n, p)  
         pmf = pd.DataFrame.from_dict(px, orient='index', columns=['P(X)'])  
         plt.figure(figsize=(6,4))  
         plt.bar(pmf.index, pmf['P(X)'])  
         plt.title(f'X~B(n={n}, p={p})')  
         plt.xlabel('X')  
         plt.ylabel('P(X)')  
         plt.show()  
         return pmf
```

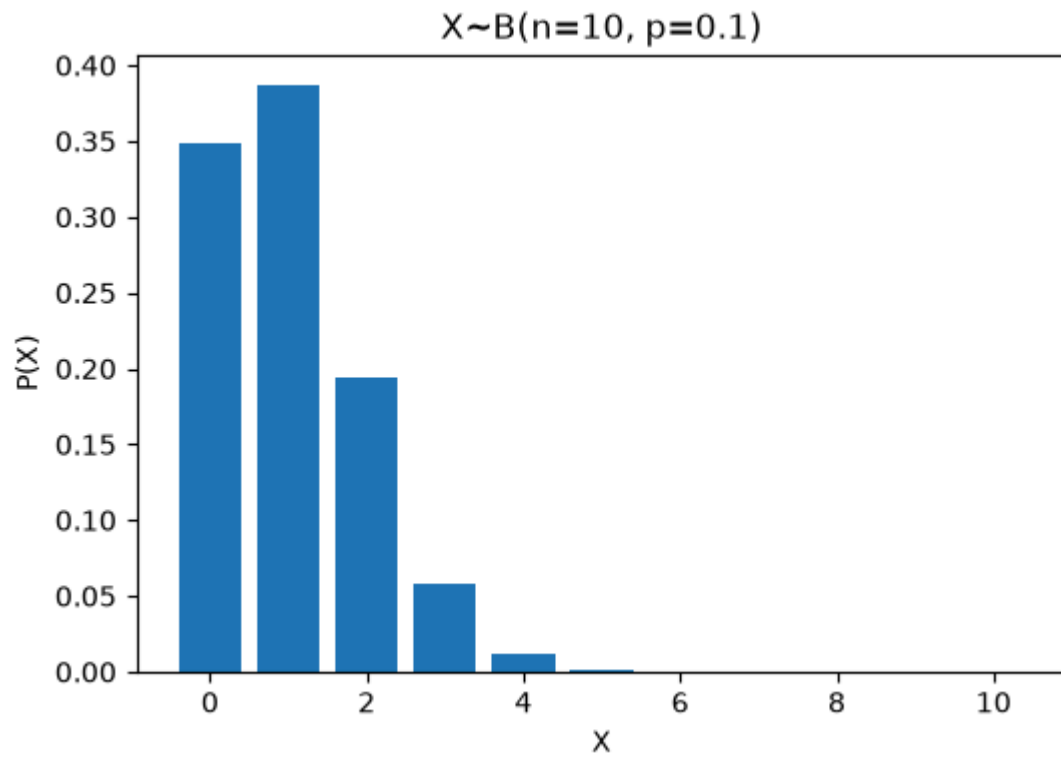
```
In [55]: draw_binom_pmf(n=10, p=0.5)
```



Out[55]:

	P(X)
0	0.000977
1	0.009766
2	0.043945
3	0.117188
4	0.205078
5	0.246094
6	0.205078
7	0.117188
8	0.043945
9	0.009766
10	0.000977

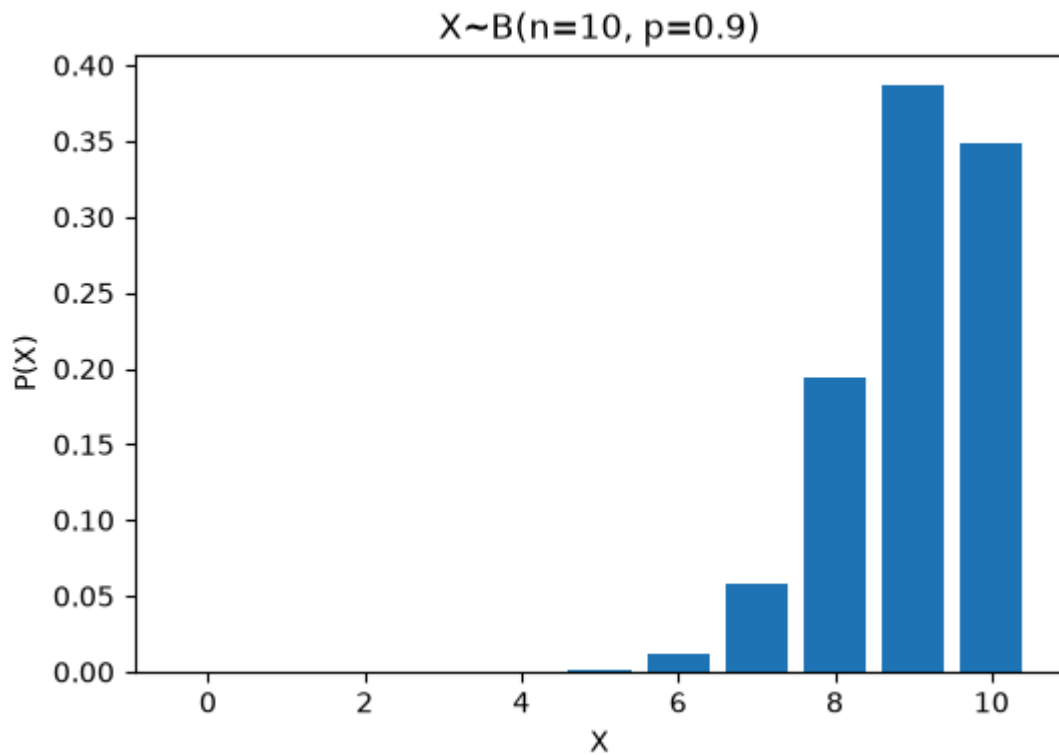
In [56]: `draw_binom_pmf(n=10, p=0.1)`



Out[56]:

	P(X)
0	3.486784e-01
1	3.874205e-01
2	1.937102e-01
3	5.739563e-02
4	1.116026e-02
5	1.488035e-03
6	1.377810e-04
7	8.748000e-06
8	3.645000e-07
9	9.000000e-09
10	1.000000e-10

In [57]: `draw_binom_pmf(n=10, p=0.9)`



Out[57]:

	P(X)
0	1.000000e-10
1	9.000000e-09
2	3.645000e-07
3	8.748000e-06
4	1.377810e-04
5	1.488035e-03
6	1.116026e-02
7	5.739563e-02
8	1.937102e-01
9	3.874205e-01
10	3.486784e-01

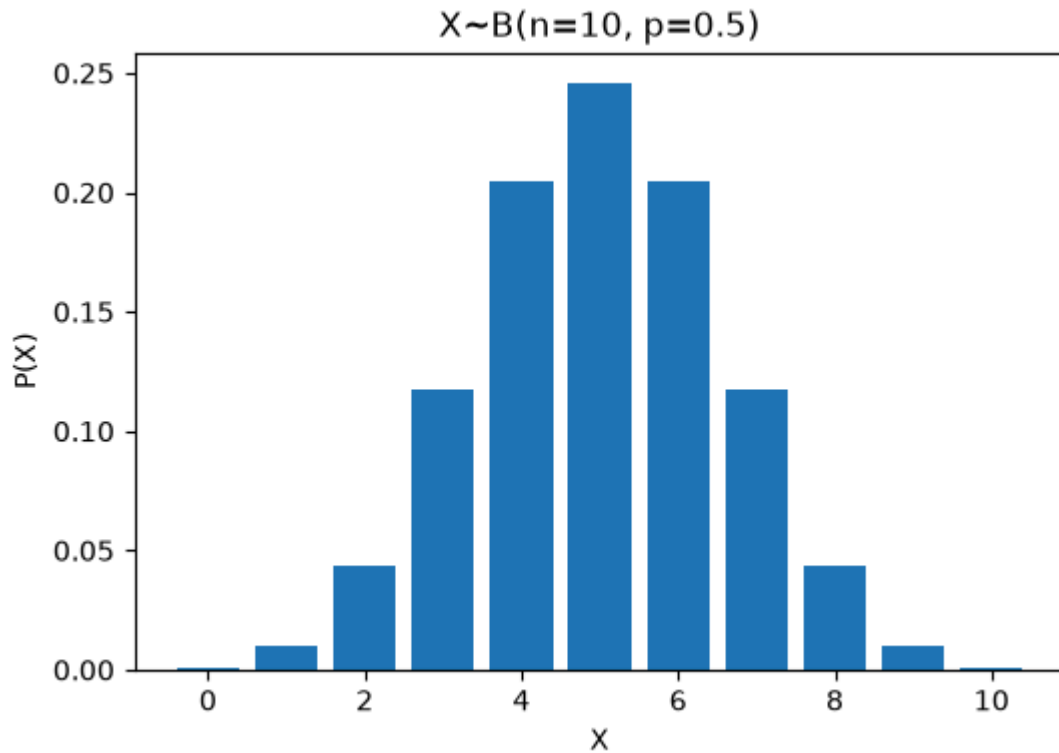
이항분포의 기대값과 분산

- 기대값($E(X)$) = 시행회수(n) * 성공확률(p)
- 분산($V(X)$) = 시행회수(n) * 성공확률(p) * 실패확률($1-p$)

```
In [58]: n, p = 10, 0.5
Ex = n*p
Vx = n*p*(1-p)
print(f'X~B(n={n}, p={p})의 E(X)={Ex}, V(X)={Vx}')
```

$X \sim B(n=10, p=0.5)$ 의 $E(X)=5.0$, $V(X)=2.5$


```
In [59]: pmf = draw_binom_pmf(n=10, p=0.5)
pmf
```



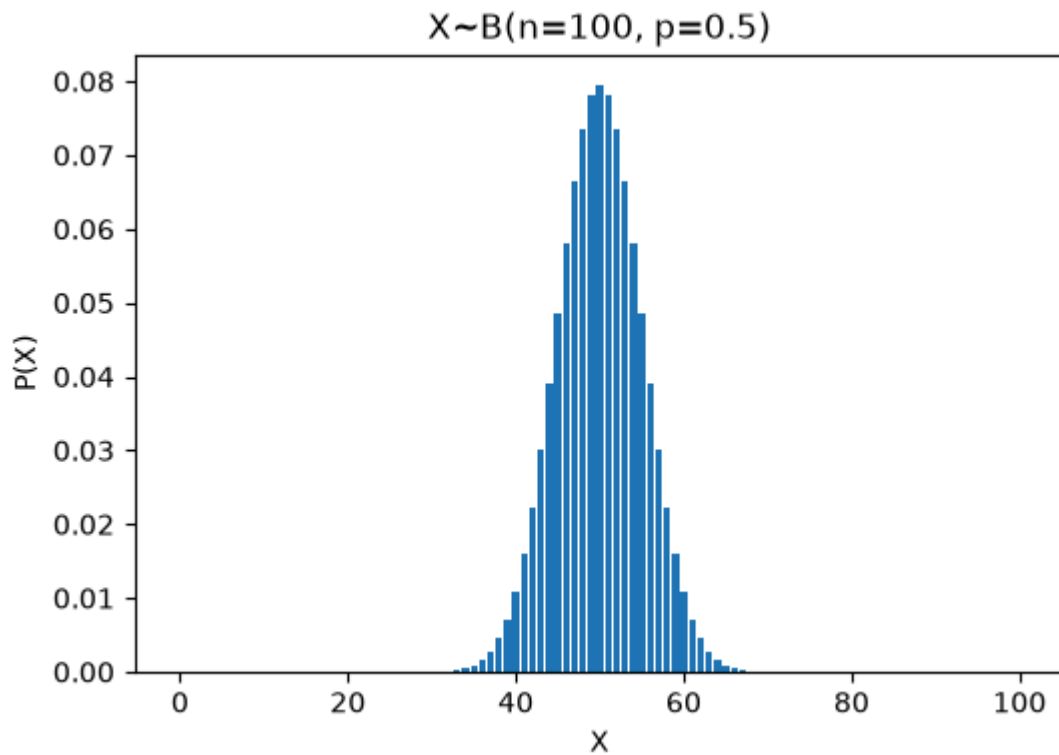
```
Out[59]:
```

	P(X)
0	0.000977
1	0.009766
2	0.043945
3	0.117188
4	0.205078
5	0.246094
6	0.205078
7	0.117188
8	0.043945
9	0.009766
10	0.000977

```
In [60]: # 기대값
Ex2 = np.dot(pmf.index, pmf['P(X)'])
# 분산 : E(x^2) - E(X)^2
Vx2 = np.dot(pmf.index**2, pmf['P(X)']) - Ex2**2
Ex2, Vx2
```

```
Out[60]: (5.000, 2.500)
```

```
In [61]: draw_binom_pmf(n=100, p=0.5)
```



Out[61]:

	P(X)
0	7.888609e-31
1	7.888609e-29
2	3.904861e-27
3	1.275588e-25
4	3.093301e-24
...	...
96	3.093301e-24
97	1.275588e-25
98	3.904861e-27
99	7.888609e-29
100	7.888609e-31

101 rows × 1 columns

Scipy.stats의 binom() 이용

```
In [62]: from scipy import stats
n,p = 10, 0.5
rv = stats.binom(n,p)
rv.pmf(0), rv.pmf(6)
```

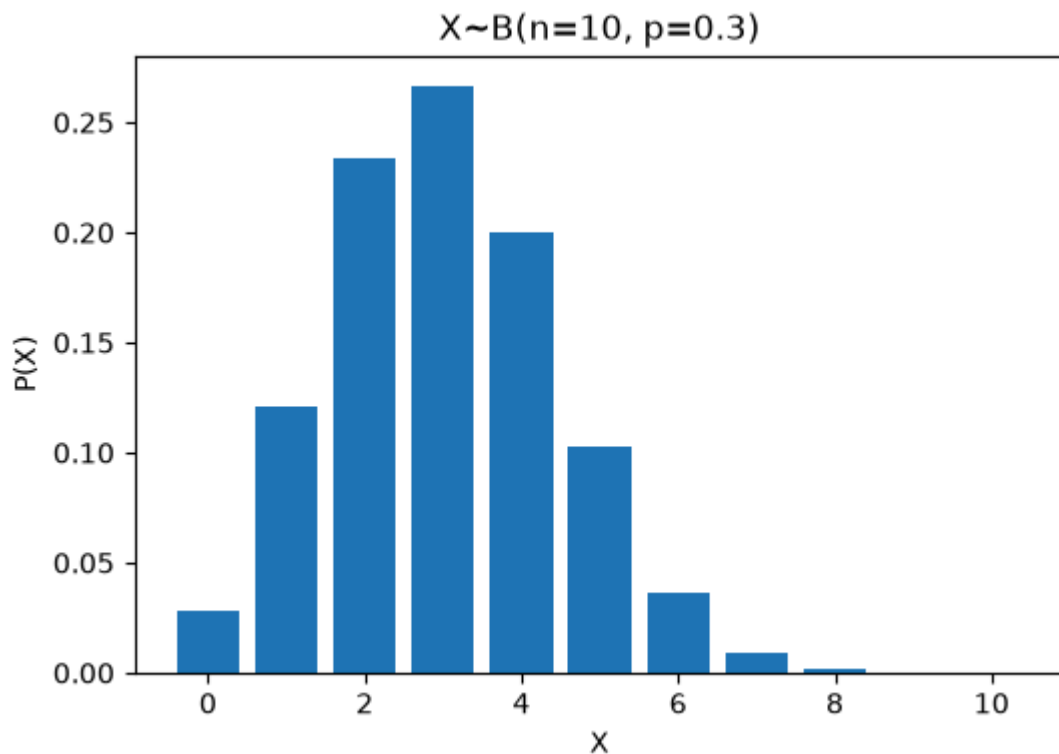
Out[62]: (0.001, 0.205)

```
In [63]: rv.pmf(np.arange(n+1))
```

```
Out[63]: array([0.001, 0.01 , 0.044, 0.117, 0.205, 0.246, 0.205, 0.117, 0.044,
               0.01 , 0.001])
```

```
In [64]: # scipy의 stats.binom() 사용
def draw_binom_pmf2(n, p):
    rv = stats.binom(n, p)
    x_set = np.arange(n+1)
    px = rv.pmf(x_set)
    plt.figure(figsize=(6,4))
    plt.bar(x_set, px)
    plt.title(f'X~B(n={n}, p={p})')
    plt.xlabel('X')
    plt.ylabel('P(X)')
    plt.show()
    pmf = pd.Series(px, name='P(X)')
    return pmf
```

```
In [65]: draw_binom_pmf2(n=10, p=0.3)
```



```
Out[65]: 0      0.028248
         1      0.121061
         2      0.233474
         3      0.266828
         4      0.200121
         5      0.102919
         6      0.036757
         7      0.009002
         8      0.001447
         9      0.000138
        10      0.000006
        Name: P(X), dtype: float64
```

```
In [66]: n,p = 10, 0.5
rv = stats.binom(n, p)
m, v, s, k = rv.stats(moments='mvsk') # 'm':mean, 'v':variance, 's':skewness, 'k'
print(f'mean:{m}, variance:{v}, skewness:{s}, kurtosis:{k:.3f}')
```

mean:5.0, variance:2.5, skewness:0.0, kurtosis:-0.200

```
In [67]: rv = stats.binom(n=10, p=0.5)
print(f'mean:{rv.mean():.3f}, var:{rv.var():.3f}, median:{rv.median():.3f}')
```

mean:5.000, var:2.500, median:5.000

연속형 확률분포

- 정규분포
- t분포
- 카이제곱분포
- F분포

numpy로 연속형 확률분포에 성질 확인을 위한 함수들 정의

In []:

In []:

In []:

정규분포

$X \sim N(\mu, \sigma^2)$

In []:

numpy로 정규분포함수 정의

In []:

In []:

In []:

In []:

In []:

scipy.stats의 norm() 함수

In []:

In []:

In []:

In []: